LegalLens

A Private, Fully On-Device Legal Document Triage Agent

Complete Solution Documentation

ZeroDay OSS Hacktoberfest 2026, IIIT Dharwad

Problem Statement PS2: Agentic AI — Private Legal Document Triage Agent

Team TH3_B0Tz · October 2026 · v1.0

LegalLens is for preliminary document understanding and issue spotting only. It is not a lawyer, gives no legal advice, and never decides whether a clause is valid, legal or enforceable.

1 Executive Summary

LegalLens reads a rental, employment or service agreement **on the user's own machine**, explains clauses in plain language, and flags obligations, penalties, deadlines, termination terms and ambiguities. Every claim is tied to a quoted passage from the document and checked by deterministic code. When the document does not say something — or the question needs a lawyer — LegalLens says so instead of guessing. Nothing the user uploads ever leaves the device.

The system is built on one design rule: **the model proposes, code disposes.** A local edge-class Gemma model drafts explanations as structured JSON with verbatim quotes; plain code then verifies every quote against the source document, every number and date in every explanation, and even the summary itself. Verdict questions such as “is this enforceable?” are refused by a code classifier before the model is ever called. Privacy is proven, not promised: the full analysis runs with the network adapter disabled, a connection monitor records zero outbound calls, and a dependency audit backs both up.

Item	Value
Track / problem statement	PS2 — Agentic AI: Private Legal Document Triage Agent
Core model	Gemma 4 E4B (edge-class), 4-bit Q4_K_M GGUF, ~4–5 GB
Runtime	llama.cpp llama-server on 127.0.0.1 (loopback only)
Language / deps	Python 3.11+, pinned requirements
Retrieval	Hybrid BM25 + local MiniLM-class embeddings, FAISS index
Privacy proof	3 layers: offline run + connection monitor + dependency audit

Item	Value
Licence	MIT (project code); model per its official card
Status	Lane A landed (20/20 tests); lanes B, C, D in progress

2 Problem Statement and Scope

2.1 The problem

Legal documents such as rental agreements, employment contracts, legal notices and service agreements contain complex terminology and clauses that ordinary people find hard to understand, and professional help for every first question is not always practical. Generic chatbots may invent clauses, answer from general knowledge and sound certain — and uploading a private contract to a cloud tool is itself a privacy risk.

2.2 In scope

- Languages: English.
- Formats: text-based PDF and plain text (.txt), one file at a time.
- Document types tested: rental / lease agreements and employment contracts (service agreements if time allows).
- Outputs: summary with citations, prioritised findings (obligations, rights, deadlines, penalties, termination, ambiguities, risks), clause-by-clause explanations with the source passage beside each, document-grounded Q&A, and an escalation panel.

2.3 Out of scope (stated honestly)

- OCR for scanned PDFs, multilingual explanations, two-version comparison, in-PDF highlighting (all optional per the rules; never penalised if absent).

2.4 Explicit non-goals

- Judging validity or enforceability; naming applicable laws as conclusions; predicting court outcomes; recommending “sign” or “don't sign”.
- Any cloud call during document processing.

3 Technology Stack

Layer	Choice	Why
Model	Gemma 4 E4B (edge-class), 4-bit Q4_K_M GGUF (~4–5 GB)	Largest edge-class variant that fits a 16 GB laptop; Apache-2.0 (confirmed on the official model card at H0). Fallback: E2B
Runtime	llama.cpp llama-server on 127.0.0.1	Local inference, no cloud; OpenAI-compatible endpoint applies Gemma's chat template (raw /completion does not — Gemma degrades without it)
Language	Python 3.11+, pinned requirements	Reproducibility
Ingestion	pypdf first, pdfplumber for layout-heavy pages	Local PDF/.txt parsing; clause-aware chunking with stable passage IDs (p3.c2)

Layer	Choice	Why
Retrieval	BM25 (rank_bm25) + local MiniLM-class embeddings, FAISS index	Hybrid: exact legal terms + paraphrase matching, all local
Vector store	FAISS, persistent local directory, telemetry disabled	Lean, battle-tested, in-process; nothing leaves the device
UI	Gradio on 127.0.0.1, analytics off, no CDN or remote fonts	Source passage shown beside every explanation
Testing	pytest (deterministic: verifier, safety rules, chunker) + bench scripts	CI runs green without model weights or network

Telemetry kill-switches (set in code, not just documented)

```
HF_HUB_OFFLINE=1   TRANSFORMERS_OFFLINE=1   HF_HUB_DISABLE_TELEMETRY=1
ANONYMIZED_TELEMETRY=False        # Chroma
GRADIO_ANALYTICS_ENABLED=False    # Gradio
STREAMLIT_BROWSER_GATHER_USAGE_STATS=false   # Streamlit
DO_NOT_TRACK=1
```

4 System Architecture

Six stages, all on the user's device. **Design rule: the model proposes, code disposes.** Gemma reads and explains; deterministic code checks every quote, every number and every refusal rule. The model never certifies its own output.

#	Stage (lane)	What happens
1	INGEST (B)	pypdf / pdfplumber extract text + page map; clause-aware chunker splits on numbered clauses (4., 4.1), headings, Article/Clause markers; passages get stable IDs (p3.c2) and character offsets; warnings for empty/corrupt/encrypted/scanned files
2	INDEX (B)	BM25 plus local embeddings index the passages into a local vector store on disk
3	ANALYSE (A)	Gemma on 127.0.0.1 only; document text enters prompts as DATA inside delimiters, never instructions; output is JSON findings (kind, plain_text, quote, passage_id, label, confidence)
4	VERIFY (C)	Deterministic code, no model: quote must be a substring of the cited passage (after normalisation); near-match ≥ 0.95 repaired and flagged; numbers/dates in an explanation must appear in its quote; failures dropped or marked unverified
5	SAFETY (C)	Labels (STATED / INTERPRETATION / CONCERN) with code-side downgrade; ambiguity and jurisdiction flags; “document is silent”; verdict refusal in code; escalation templates; fixed disclaimer
6	UI (D)	Gradio on 127.0.0.1 only: summary · risk-first findings · clause view with the source passage beside each explanation · Q&A · escalation panel

Runtime network policy

Network at normal run time: NONE. Setup-time downloads only (model weights, packages, optional embedding model) — explicitly allowed by the rules provided no private document content is sent, which the design makes impossible.

5 Grounding: Evidence on Every Claim

Grounding carries a 15-point criterion and a score cap. Six mechanisms deliver it:

1. **Passage IDs everywhere.** The model only ever sees passages with IDs and must cite one.

2. **Verbatim quote check.** After normalisation (Unicode NFKC, whitespace collapse, curly-quote/dash unification, case-fold), the quote must be a substring of the cited passage. Near-match (≥ 0.95 similarity, e.g. a dropped comma) is repaired with the true source span and flagged quote_corrected; anything else is dropped and counted in eval as a verifier rejection.
3. **Number/date consistency.** Every amount, day-count, percentage and date in an explanation must appear in its quote. Catches "30 days" explained as "one month" — the most harmful plain-language distortion.
4. **"The document is silent."** If the best retrieved passage scores below threshold and nothing verifies, the reply is the fixed text "The document does not appear to say anything about this" — never general knowledge.
5. **Coverage report.** Clause types found and not found ("no termination clause found"), so users see what is missing, not just what is present.
6. **The summary is verified too.** The summary is generated from verified findings only; every sentence carries passage IDs; a sentence whose numbers do not match its citation is dropped, not repaired. The most claim-dense artifact on screen is never the unverified surface.

6 Uncertainty, Refusal and Escalation

6.1 Separating text from interpretation

Label	Meaning	UI treatment
STATED	The quote directly and fully supports the explanation, zero inference	Plain, quote beside it
INTERPRETATION	A reasonable reading that needs inference beyond the literal words	Marked "our reading"; never used for a verdict
CONCERN	Something that may deserve attention (one-sided, vague, unusual)	Marked "worth asking about", with a question to ask

The prompt asks Gemma to pick the label, and code downgrades it when evidence is weak — a finding whose explanation adds a fact not in its quote cannot be STATED.

6.2 Verdict refusal — in code, not the prompt

Questions like "is this legal?", "is this enforceable?", "should I sign?" are classified by a deterministic classifier and refused before the model is ever called. The refusal is specific: which clause, what to ask a qualified legal professional, what facts to bring. Prompt wording is only a backstop for phrasings the classifier misses.

> *"I cannot determine that. This depends on your jurisdiction and specific circumstances. Consider asking a qualified legal professional about <topic>, and bring the exact clause text with you."*

6.3 Ambiguity and jurisdiction

- Vague terms ("reasonable", "sole discretion", "from time to time"), blank placeholders and conflicting clauses become ambiguity findings with flags.

- Jurisdiction-dependent topics (deposit size and return, notice periods, non-compete, probation, overtime, stamp duty, rent increase) are flagged “this depends on where you are” with no legal conclusion.

6.4 Prompt-injection defence

7. Document text enters prompts only inside clearly delimited data blocks, with a system instruction that its contents are untrusted text.
8. A pre-scan flags instruction-like strings (“ignore previous instructions”, “you are now”, “output the following”) and surfaces them: “this document contains text that looks like an instruction to an AI; it was ignored”.
9. Output is a fixed JSON schema — injected text cannot add fields or change the response shape.
10. Verdict refusal happens before the model is called, so an injected instruction cannot talk the system into a legal conclusion.

Tested with injected.pdf: the analysis output contains no trace of the injected instruction (asserted in the test suite).

7 Retrieval: BM25 + Embeddings + FAISS

7.1 Why retrieve at all

Documents run 30+ pages; the 8k context cannot hold them, and long contexts make small models paraphrase instead of quote. At upload the document is indexed once; per question only the top-3 clauses enter the prompt. Retrieval is the funnel that makes both grounding and the ~30-second latency budget possible.

7.2 Two retrievers that fail in opposite ways

	BM25	Dense embeddings
What it is	Classic exact-word ranking (rare-word weighting, length normalisation)	Local MiniLM-class model maps clauses and the question to vectors; nearest vectors win
Best at	Precise legal terms: “security deposit”, “non-compete”, “lock-in period”	Paraphrases: “can I get my money back if I leave early?” → deposit-refund clause
Fails when	User's words don't match the document's words	Rare exact legal terms a small embedding model under-saw
Cost	Pure math over word counts — deterministic, instant, no download	~100 MB model downloaded at setup, loaded from disk at runtime

Why both: they fail in opposite ways. Legal language is keyword-heavy; user questions are paraphrase-heavy. Rankings are merged (reciprocal-rank fusion) and the top-3 passages go to the prompt.

7.3 FAISS — the local vector index

FAISS (Facebook AI Similarity Search) stores the clause vectors and returns the top-k nearest in milliseconds — in-process, open source, no network. For a 3-page rental a brute-force scan would do; FAISS costs nothing and serves unbounded documents with the same code path. Similarity only: all semantic quality comes from the embedding model; FAISS is pure speed.

7.4 The silent path is retrieval-driven

If the best merged score is below threshold, code returns “the document does not appear to say anything about this” — the model never sees the question. Code decides the document probably does not cover the topic; the model never gets the chance to invent.

8 Model Choice: Why This Gemma Variant

The rules mandate an edge-class Gemma running on-device doing meaningful document understanding — either generation (Gemma 3n or Gemma 4 E2B/E4B) is accepted. The variant was derived from constraints, then locked after measurement at H1:

Constraint	Rules out	Survives
Rules: edge-class Gemma, local, private	9B/27B+; any cloud inference	E2B / E4B
Hardware: 16 GB laptop, RAM shared with OS + UI + embeddings	Weights > 6–8 GB	E4B 4-bit ≈ 4–5 GB
Task: character-exact quotes, JSON schema, refusals	Weak instruction-following → more verifier rejections	E4B over E2B
Latency: ~30 s per Q&A on CPU	Big prompts / models	top-3 retrieval + E4B

- Why Gemma 4 generation: same accepted E-class, reported Apache-2.0 (confirmed on the official model card at H0), better JSON reliability at equal RAM.
- Why 4-bit (Q4_K_M): RAM/quality sweet spot for extraction-style tasks.
- **Fallback ladder (measured, not guessed):** context 8192 → 4096 → Q3_K_S → E2B. Every switch re-measured; results committed to bench/results/.

> *“We didn't pick a variant and justify it — we derived it from the rules, the hardware and the task, then locked it after measurement.”*

9 Privacy Architecture and Proof

Privacy carries a 15-point criterion and a score cap. Judges look for the absence of connections (encrypted payloads are invisible), so the proof needs three independent layers:

Layer	Claim	Method
1	Cannot connect	Full pipeline run with networking disabled: adapter off / Airplane mode (Windows, captured with Get-NetConnectionProfile), container --network none, or WSL2 namespace. Loopback still works, so the run is unchanged
2	Does not try	Connection monitor (ss / tcpdump / Get-NetTCPConnection) around a normal run; expected: zero non-loopback connections. Raw logs saved to docs/examples/
3	Has no code that connects	privacy/audit_deps.py greps code and installed packages for HTTP clients, update checks, telemetry; every hit reviewed and recorded in THREAT_MODEL.md

9.1 Supporting controls

- Model server and vector store bind to 127.0.0.1 or run in-process.
- Weights and the embedding model are setup-time downloads, loaded from disk at runtime; HF_HUB_OFFLINE=1 and friends are set in code, making hub access impossible even by accident.

- Session artifacts live in one documented directory with a “clear session” control; a temp-file audit confirms nothing document-derived lands anywhere else.
- Logs hold timings, counts and error codes only — verified by grepping logs for a distinctive phrase from a test document.

9.2 Threat model summary

- **Protected:** the file, extracted text, chunks, embeddings, summaries, answers, names, metadata, and the user's questions.
- **Defended against:** remote services, telemetry, shared-machine leakage, instruction injection inside documents.
- **Not claimed:** a compromised OS, a malicious model file, physical or screen access.

10 Module Reference (Ho Contracts)

Core types (legallens/types.py)

```
Passage:   id ("p3.c2"), page, heading, text, start, end
Document:  doc_id, source_name, passages, extraction_warnings
Finding:   id, kind, party, plain_text, quote, passage_id, label,
           confidence, flags, escalation_note
  kind:  obligation | right | deadline | penalty | termination | ambiguity | risk
  label: STATED | INTERPRETATION | CONCERN
Analysis:  summary, summary_citations, findings, coverage
Reply:     text, citations[{passage_id, quote}], silent, refused, escalation_note
VerifyResult: ok, reason, repaired_quote, pruned_summary
```

Function signatures

```
# Lane A (landed)
analyze(doc: Document) -> Analysis          # extract -> verify -> summary -> coverage
ask(doc: Document, question: str) -> Reply  # route in code -> retrieve -> answer -> verify

# Lane B (pending)
ingest(path: str) -> Document
retrieve(doc, query: str, k=3) -> list[tuple[Passage, float]]

# Lane C (pending)
verify(item: Finding | Reply | Analysis, doc) -> VerifyResult
classify_question(q) -> explain | document_question | verdict_request | off_topic

# Lane D (pending)
load_sample(name: str) -> str
eval_set() -> list[EvalCase]
```

Module boundaries are import boundaries: analyze.py auto-swaps to B's and C's real modules the moment they are importable; until then clearly-flagged permissive stubs fill in, so lane A is fully testable offline.

11 Test and Evaluation Plan

11.1 Current test suite (20/20 passing, no model or network needed)

- JSON client handling: fences, chatty replies, top-level arrays, retry loop with error feedback, schema-check-driven retries, give-up after max retries.
- Analysis shapes: findings, summary citations, coverage, stub-verifier flagging.
- Safety paths: verdict questions refused in code (model never called — asserted via empty call log), “should I sign” refused, silent path, injected document contained (no “fully legal” leak).
- Type contracts: field names match the H0 agreement exactly.

11.2 Planned suites (lanes B–D)

- Chunker: numbered clauses split correctly; passage IDs stable across runs; offsets match the text.
- Verifier: fake quote rejected; curly-quote/whitespace variants accepted; “30 days” vs “one month” caught.
- Safety rules: verdict phrasing never appears in output; escalation specific; disclaimer always present.
- Eval: citation accuracy, silent-case correctness, refusal-case correctness, ambiguity flagging, injection results.
- Robustness: malformed inputs (empty, corrupt, huge, password-protected, scanned) — clear message, never a crash; 30+ page documents finish with merged findings.
- Speed and memory: tokens/sec, peak RAM, seconds per page, reported with exact hardware.

11.3 Metrics reported honestly

Quote-verification pass rate before and after repair; findings dropped; citation accuracy; silent-case and refusal-case correctness; label distribution; latency per document and per question; peak RAM; model size and quantisation; known failures with examples — reporting weaknesses scores better than hiding them.

12 Implementation Status

Lane	Scope	Status
A — Gemma + analysis pipeline	llm.py, prompts/, analyze.py: local client, JSON extraction with retries, summary with citations, Q&A with code-side refusal	Landed — 20/20 demo
B — ingestion, retrieval, privacy	ingest.py, retrieve.py, privacy/: parsing, chunking, hybrid index, offline run, kill-switches, dependency audit, threat model	Pending — perm import boundary
C — grounding + safety	verify.py, safety.py, injection.py: quote/number verifier, labels, flags, silence logic, refusal, escalation, output filter	Pending — refus in analyze.py unt
D — UI, bench, docs, demo	app/, bench/, data/samples/, docs, README, CI, demo recording, submission	Pending

13 Honest Limitations

11. English only; text-based PDF and .txt; no OCR for scans (detected and reported, never guessed).
12. Tested scope: rental and employment agreements; service agreements if time allowed.
13. Retrieval bounds Q&A quality — a relevant clause ranked low yields a silent answer, never a wrong one. A deliberate trade: silence is honesty; a wrong citation is fabrication.
14. Small-model wording is kept deliberately terse; the number/date check hides any explanation that drifts.

15. Privacy covers the software, not a compromised OS or a malicious model file — scope stated in the threat model.

14 Main Technical Contribution

> *"A verification-first architecture for small local models: we don't trust Gemma to be right, we make it provably grounded — quotes, numbers and the summary are all checked by deterministic code, so a 4-bit edge model can't fabricate. The refusal logic lives in code, not the prompt, so it can't be talked out of it — not even by instructions hidden in the document. And because it's edge-class, the whole thing runs on a laptop with the network adapter off."*

Why this matters: the three failure modes the rubric penalises most — fabricated citations, unsupported legal conclusions, and content leaving the device — are made impossible by construction rather than discouraged by prompting. The verifier and the chunker are reusable components in their own right.

15 How to Reproduce

Setup (needs network once)

```
git clone <repo-url> && cd legallens
python -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
# accept the Gemma licence on the official model card, then:
huggingface-cli download <gemma-4-e4b-gguf-repo> \
    --include "*Q4_K_M*" --local-dir models
```

Run (offline)

```
llama-server -m models/<gemma-4-e4b-Q4_K_M>.gguf \
    --host 127.0.0.1 --port 8080 -c 8192

python smoke.py            # offline stub demo (no server needed)
python smoke.py --real     # full flow against the local model
pytest tests/ -q           # deterministic suite, no model/network
```

The README separates "Setup (needs network once)" from "Run (offline)"; the offline claim is verified by disabling the network adapter before the run and capturing the connection monitor output.

16 Demo Script (~5 minutes)

16. **Problem (30 s):** People sign rentals and jobs without understanding them; generic chatbots invent clauses and sound certain; uploading a private contract to the cloud is a privacy risk.
17. **Privacy proof (60 s):** Disable networking; run the full analysis live; show the connection monitor with zero outbound connections.
18. **Live on an unseen document (2.5 min):** Upload an agreement the team did not tune on. Show summary and top findings with the quoted passage beside each explanation and its label. Ask a question the document cannot answer → "the document does not say". Ask "is this enforceable?" → declined with escalation. Open injected.pdf → instruction flagged and ignored.
19. **Results (45 s):** Eval table: citation accuracy, verifier rejections, silence and refusal cases, speed and RAM on the demo laptop, plus one honest failure.

20. **Close (15 s):** Gemma runs on the device, code checks every quote, nothing leaves the machine, and the tool knows its limits.

17 Licence and Attribution

- Project code: MIT (LICENSE in the repository root).
- Model: Gemma — licence per its official model card (see docs/MODEL_SETUP.md for acquisition steps and the recorded SHA).
- Sample documents: written by the team, safe to redistribute with the project.
- Optional benchmark CUAD (CC BY 4.0): cited in any results table that uses it; never committed to the repository.
- No user documents, credentials or model weights are committed (gates G4, G5).